

OI 2024 模拟赛

OI 2024 Simulation

2024 年 9 月 15 日

题目名称	三角形	子段异或	三等分	二叉树遍历
目录	triangle	xor	three	traversing
可执行文件名	triangle	xor	three	traversing
输入文件名	triangle.in	xor.in	three.in	traversing.in
输出文件名	triangle.out	xor.out	three.out	traversing.out
每个测试点时限	1.0 秒	1.0 秒	2.0 秒	6.0 秒
内存限制	512 MB	512 MB	512 MB	512 MB
子任务数目	4	5	6	9
测试点是否等分	否	否	否	否

提交源程序文件名

对于 C++ 语言	triangle.cpp	xor.cpp	three.cpp	traversing.cpp
-----------	--------------	---------	-----------	----------------

编译选项

对于 C++ 语言	-lm -O2	-lm -O2	-lm -O2	-lm -O2
-----------	---------	---------	---------	---------

【注意事项（请仔细阅读）】

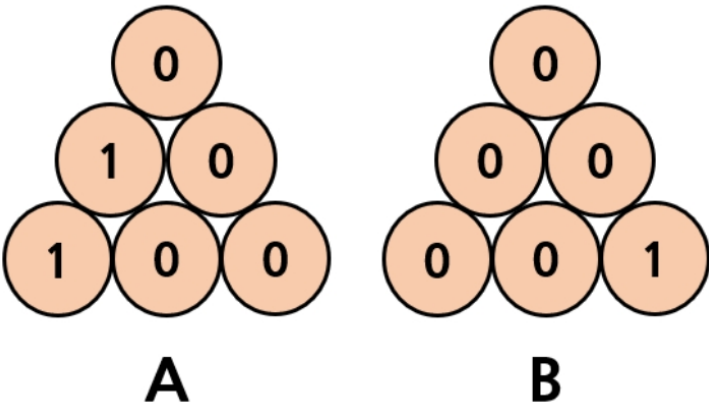
1. 选手提交的源程序请直接放在个人目录下，无需建立子文件夹；
2. 文件名（包括程序名和输入输出文件名）必须使用英文小写。
3. C++ 中函数 `main()` 的返回值类型必须是 `int`，值必须为 0。
4. 对于因未遵守以上规则对成绩造成的影响，相关申诉不予受理。
5. 若无特殊说明，结果比较方式为忽略行末空格、文末回车后的全文比较。

6. 程序可使用的栈空间大小与该题内存空间限制一致。
7. 在终端中执行命令 `ulimit -s unlimited` 可将当前终端下的栈空间限制放大，但你使用的栈空间大小不应超过题目限制。
8. 若无特殊说明，每道题的**代码大小限制为 100KB**。
9. 若无特殊说明，输入与输出中同一行的相邻整数、字符串等均使用一个空格分隔。
10. 输入文件中可能存在行末空格，请选手使用更完善的读入方式（例如 `scanf` 函数）避免出错。
11. 直接复制 PDF 题面中的跨页样例，数据将带有页眉页脚，建议选手直接使用对应目录下的样例文件进行测试。
12. 使用 `std::deque` 等 STL 容器时，请注意其内存空间消耗。
13. 请务必使用题面中规定的的编译参数，保证你的程序在本机能够通过编译。此外**不允许在程序中手动开启其他编译选项**，一经发现，本题成绩以 0 分处理。

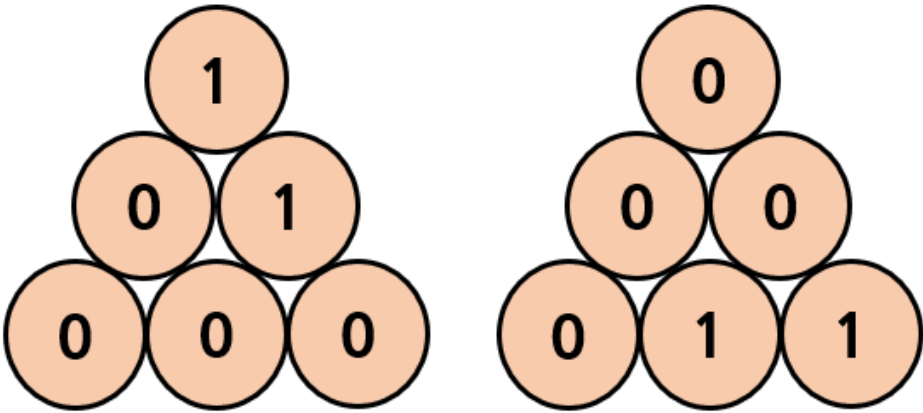
三角形 (triangle)

【题目描述】

有一种用硬币摆成的三角形，第一行有 1 个硬币，第二行有 2 个硬币，第三行有 3 个硬币，...，第 n 行有 n 个硬币，如下图所示，其中 0 表示正面，1 表示反面。

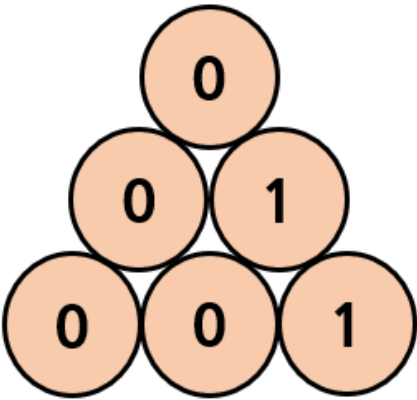


你可以旋转该三角形任意次，如下图是三角形 A 可以通过旋转变成的形态。



同时你也可以使三角形左右对称。如下图是三角形 A 对称后的形态。

定义两个三角形差值是两个三角形重叠时不同位置的数目。例如将三角形 A 和 B 重叠，第二行第一个，第三行第一个、第三个都不同。此时差值为 3。如果将 A 逆时针旋



转 120 度（第二张图右边的三角形）和 B 重叠，那么只会有一个位置不同，差值为 1。

给出两个大小为 N 的三角形 A 和 B，你可以任意的旋转和对称 A，求 A 旋转和对称后和 B 最小的差是多少。

【输入格式】

从文件 *triangle.in* 中读入数据。

第一行输入一个整数 N，表示 A 和 B 的大小。

在接下来的 N 行中，第 i 行有 i 个整数，每个整数都是 0 或者 1，表示三角形 A。

在接下来的 N 行中，第 i 行有 i 个整数，每个整数都是 0 或者 1，表示三角形 B。

【输出格式】

输出到文件 *triangle.out* 中。

输出一行一个整数，表示通过旋转和翻转能使 A 和 B 最小的差是多少。

【样例输入】

```
3
0
1 0
1 0 0
0
0 0
0 0 1
```

【样例输出】

1

【样例输入】

```
4
0
1 1
1 0 0
1 0 0 0
0
0 0
0 0 1
1 1 1 0
```

【样例输出】

0

【样例输入】

```
4
0
1 0
0 0 1
1 1 0 0
0
0 1
0 0 0
0 1 1 1
```

【样例输出】

2

【测试点约束】

对于所有测试点： $1 \leq N \leq 10$,A 和 B 中仅包含 0 和 1
每个测试点的具体限制见下表：

Subtask	Points	附加条件
1	5	A 中所有数字相同
2	10	$N \leq 2$
3	40	保证存在最优解只需要旋转
4	45	无附加条件

子段异或 (xor)

【题目描述】

给你一个 01 串 S ，下标范围为 $[1, |S|]$ ， $f_S(l, r)$ 表示将 S 中下标从 l 到 r 的 01 串取出，按二进制转化为整数。

请你求出

$$\max_{1 \leq l_1 \leq r_1 \leq n, 1 \leq l_2 \leq r_2 \leq n} f_S(l_1, r_1) \text{ xor } f_S(l_2, r_2)$$

的二进制表示。其中 xor 表示二进制按位异或。

【输入格式】

从文件 *xor.in* 中读入数据。第一行输入一个 T ，表示数据组数。

对于每组数据，第一行输入一个整数 n ，表示 S 串的长度。

第二行输入一个长度为 n 的字符串，表示 S ，保证字符串内只包含 0 和 1

【输出格式】

输出到文件 *xor.out* 中。

对于每组数据，输出一行一个 01 串，表示最大异或值的二进制表示。请注意不要前导 0。

【样例输入】

```
4
3
010
5
10101
5
00100
5
11111
```

【样例输出】

```
11
11111
110
11110
```

【样例输入】

```
4
2
00
2
01
2
10
2
11
```

【样例输出】

```
0
1
11
10
```

【测试点约束】

对于所有测试点： $1 \leq T \leq 100$ $2 \leq N \leq 10^7$ N 的和不超过 10^7 。
每个测试点的具体限制见下表：

Subtask	Points	附加条件
1	17	$N \leq 30, N$ 的和 ≤ 300
2	20	$N \leq 200, N$ 的和 ≤ 2000
3	13	$N \leq 3000, N$ 的和 ≤ 30000
4	12	$N \leq 2 \times 10^5, N$ 的和 $\leq 2 \times 10^6$
5	38	无附加条件

三等分 (three)

【题目描述】

给你一个长度为 N 的序列，保证 N 是 3 的倍数。序列中每个数字都是 $[1, M]$ 的整数。

Alice 想把这个序列转化成由若干个三元组成的可重集合，原序列中每个元素只能使用一次，可以选择三个一样的数字或者三个连续的数字合并成一个三元组。即每个三元组都是 (x, x, x) 或者 $(x, x + 1, x + 2)$ 。

两种转化方式视为相同当且仅当合并后的三元组集合相同，即两种方法的三元组可以一一的产生数字相等的对应关系。Alice 想知道：一共有多少种不同的转化方法？由于答案过大，请你输出答案对 $10^9 + 7$ 取模的值。

【输入格式】

从文件 *three.in* 中读入数据。第一行输入两个整数 n 和 m 。

【输出格式】

输出到文件 *three.out* 中。

第二行输入 n 个整数，每个整数范围是 $[1, m]$

【样例输入】

9 4
3 4 2 4 4 2 3 3 2

【样例输出】

2

可以分为 $(2,2,2), (3,3,3), (4,4,4)$ 或者 $(2,3,4), (2,3,4), (2,3,4)$ 。

【样例输入】

6 3
1 2 3 1 2 1

【样例输出】

0

【测试点约束】

对于所有测试点： $1 \leq n \leq 5000, 1 \leq m \leq 5000, n \bmod 3 = 0, 1 \leq a_i \leq m$
每个测试点的具体限制见下表：

Subtask	Points	附加条件
1	10	$m \leq 3$
2	8	$m \leq 4$
3	10	序列中每个数字出现不超过 2 次
4	12	序列中没有 4 的倍数
5	29	$n \leq 500, m \leq 500$
6	31	无附加条件

二叉树遍历 (traversing)

【题目描述】

二叉树是一个有根树，每个结点有至多两个子结点。其中有一个结点不是任何结点的子结点，这个结点是根结点。

二叉树有三种重要的遍历方式分别被称为：前序遍历，中序遍历，后序遍历。

它们的区别在于在是左右儿子之前、之中、之后输出。若前序遍历，则先输出当前结点，再输出左子树的遍历结果，再输出右子树的遍历结果。若中序遍历，则先输出左子树的遍历结果，再输出当前结点，再输出右子树的遍历结果。若后序遍历，则先输出左右子树的遍历结果，最后再输出当前结点。

如果我们给树上每个顶点都写一个整数 x ， x 是一个 $[-1, 1]$ 的整数。 x 的值表示以该点为根的子树采用什么遍历方式。

- $x = -1$ ：该点为根的子树采用前序遍历。
- $x = 0$ ：该点为根的子树采用中序遍历。
- $x = 1$ ：该点为根的子树采用后序遍历。

现在有一个 n 个顶点的树，编号为 1 到 n 。 1 是树的根，每个结点上都记录了数字 -1 。

你需要执行如下操作：

- 1, 将结点编号在 $[l, r]$ 范围内的结点上的数字都赋值为 $x (x \in \{-1, 0, 1\})$ 。
- 2, 输出结点 i 当前在整棵树上第几个被遍历到。

【输入格式】

从文件 *traversing.in* 中读入数据。第一行输入两个整数 n, q 。分别表示树上结点的个数和询问次数。

接下来 n 行，第 i 行输入两个整数 L_i, R_i ，表示 i 号点的左儿子和右儿子编号，若没有对应的儿子，值为 0 。

接下来 q 行，每行第一个整数 t 表示操作的类型。

若 t 为 1 ，该行之后输入三个整数 l, r, x ，表示操作的区间和区间整体赋值的值。

若 t 为 2 ，该行之后输入一个整数 i ，表示查询的结点的编号。

【输出格式】

输出到文件 *traversing.out* 中。

对于每个 $t = 2$ 的查询，输出一个值在 $[1, n]$ 的整数，表示所询问顶点在当前遍历中的位置。

【样例输入】

```
5 5
3 4
0 0
5 2
0 0
0 0
2 2
1 1 3 1
2 5
1 3 3 0
2 3
```

【样例输出】

```
4
1
2
```

【测试点约束】

对于所有测试点： $1 \leq n, q \leq 10^5, 0 \leq L_i, R_i \leq n$ ，并且保证输入的是一个合法的二叉树。 $t \in \{1, 2\}, 1 \leq l \leq r \leq n, x \in \{-1, 0, 1\}, 1 \leq i \leq n$

每个测试点的具体限制见下表：

Subtask	Points	附加条件
1	10	$n, q \leq 5000$
2	5	1 类操作个数 ≤ 10
3	10	所有 1 类操作早于所有 2 类查询
4	10	给定的是一颗满二叉树
5	10	所有 1 类操作 $l = r$
6	20	1 类操作 $x \in \{-1, 1\}$ ，且给出的树是一条链
7	10	1 类操作 $x \in \{-1, 1\}$
8	10	给出的树是一条链
9	15	无附加条件